LAPORAN UJIAN TENGAH SEMESTER

Sistem Booking Lapangan Olahraga

Disusun oleh
Muhammad Assya Dili 2410511078

Pemrograman Perangkat Lunak Berorientasi Service
Fakultas Ilmu Komputer
Universitas Pembangunan Nasional Veteran Jakarta

BAB 1 - PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi mendorong berbagai sektor untuk mengadopsi solusi digital yang lebih efisien dan terstruktur. Salah satu pendekatan yang semakin populer dalam pengembangan perangkat lunak modern adalah arsitektur berbasis layanan atau Service-Oriented Architecture (SOA), khususnya dalam bentuk microservice.

Sistem Booking Lapangan Olahraga merupakan solusi digital yang dirancang untuk memudahkan pengguna dalam melakukan reservasi lapangan olahraga secara online. Sistem ini dibangun menggunakan pendekatan microservice yang memisahkan domain bisnis menjadi layanan-layanan kecil yang independen, sehingga memudahkan pengembangan, pengujian, dan pemeliharaan sistem.

1.2 Rumusan Masalah

- Bagaimana merancang dan mengimplementasikan sistem booking lapangan olahraga berbasis microservice?
- Bagaimana mengimplementasikan mekanisme autentikasi menggunakan JWT dan OAuth 2.0?
- Bagaimana mengimplementasikan API Gateway sebagai single entry point dengan JWT validation dan rate limiting?
- Bagaimana menerapkan komunikasi antar-service dalam arsitektur microservice?

1.3 Tujuan

- Membangun sistem booking lapangan olahraga berbasis microservice dengan minimal 3 service independen.
- Mengimplementasikan autentikasi JWT dengan access token (15 menit) dan refresh token (7 hari).
- Mengimplementasikan OAuth 2.0 menggunakan Google sebagai provider.
- Membangun API Gateway dengan fitur routing, JWT validation, dan rate limiting.

BAB 2 - ARSITEKTUR SISTEM

2.1 Gambaran Umum Arsitektur

Sistem dibangun menggunakan arsitektur microservice dengan 3 service independen dan 1 API Gateway sebagai single entry point. Setiap service memiliki database tersendiri sesuai prinsip database-per-service dalam microservice.

Komponen	Teknologi	Port	Database
API Gateway	Express.js (Node.js)	3000	-
Auth Service	Express.js (Node.js)	3001	auth_db
Field Service	Laravel 11 (PHP)	3002	field_db
Booking Service	Express.js (Node.js)	3003	booking_db

2.2 Justifikasi Pemisahan Service

2.2.1 Auth Service

Auth Service dipisahkan menjadi service tersendiri karena memiliki tanggung jawab tunggal yaitu manajemen identitas dan autentikasi pengguna. Dalam pendekatan monolitik, logika autentikasi tersebar di berbagai modul sehingga sulit dikelola. Dengan memisahkannya, tim pengembang dapat mengupdate mekanisme autentikasi tanpa mempengaruhi service lain. Service ini juga menjadi pusat validasi token yang digunakan oleh API Gateway.

2.2.2 Field Service

Field Service dipisahkan karena mengelola domain data lapangan yang memiliki siklus hidup berbeda dengan data booking. Data lapangan relatif statis (jarang berubah), sedangkan data booking sangat dinamis. Pemisahan ini juga memungkinkan penggunaan teknologi yang berbeda - Laravel 11 dengan PHP MVC dipilih karena kemampuannya dalam mengelola relasi database yang kompleks dengan ORM Eloquent.

2.2.3 Booking Service

Booking Service dipisahkan karena memiliki logika bisnis yang kompleks terkait transaksi pembayaran dan manajemen status booking. Service ini berkomunikasi dengan Field Service untuk melakukan lock dan release slot, mencerminkan komunikasi antar-service dalam arsitektur microservice. Pemisahan ini memungkinkan skalabilitas independen mengingat frekuensi transaksi booking dapat sangat tinggi.

2.3 Perbandingan dengan Pendekatan Monolitik

Aspek	Monolitik	Microservice
Deployment	Satu unit besar	Per service secara independen
Skalabilitas	Skala keseluruhan sistem	Skala per service sesuai kebutuhan
Teknologi	Satu stack teknologi	Bebas memilih per service
Database	Satu database bersama	Database terpisah per service
Fault Isolation	Satu error bisa down seluruh sistem	Error terisolasi per service
Kompleksitas	Lebih sederhana awal	Lebih kompleks namun scalable

BAB 3 - REST API DAN DATABASE

3.1 Desain RESTful API

HTTP Method	Semantik	Contoh Penggunaan
GET	Mengambil data	GET /fields - ambil daftar lapangan

HTTP Method	Semantik	Contoh Penggunaan
POST	Membuat data baru	POST /bookings - buat booking baru
PUT	Update seluruh data	PUT /fields/:id - update lapangan
PATCH	Update sebagian data	PATCH /slots/:id/lock - kunci slot
DELETE	Hapus data	DELETE /categories/:id - hapus kategori

HTTP Code	Makna	Contoh Kasus
200 OK	Request berhasil	Login berhasil, get data berhasil
201 Created	Data berhasil dibuat	Register berhasil, buat booking
204 No Content	Berhasil tanpa response body	Delete berhasil
400 Bad Request	Request tidak valid	Field wajib tidak diisi
401 Unauthorized	Token tidak valid/tidak ada	Akses tanpa token
403 Forbidden	Tidak punya izin	User akses endpoint admin
404 Not Found	Data tidak ditemukan	ID lapangan tidak ada
409 Conflict	Data duplikat/konflik	Email sudah terdaftar
422 Unprocessable	Validasi gagal	Format email salah
500 Server Error	Error internal server	Database error

3.2 Struktur Database per Service

3.2.1 Auth Service (auth_db)

Tabel	Kolom Utama	Keterangan
users	id, name, email, password_hash, oauth_provider, oauth_provider_id, is_oauth_user, profile_photo_url, role	Data pengguna sistem
refresh_tokens	id, user_id, token, expires_at, is_revoked, ip_address	Refresh token per pengguna
token_blacklist	id, user_id, access_token, expires_at, blacklisted_at	Token yang sudah di-logout

3.2.2 Field Service (field_db)

Tabel	Kolom Utama	Keterangan
field_categories	id, name, description	Kategori lapangan (futsal, basket, dll)
fields	id, category_id, name, description, location, price_per_hour, status	Data lapangan
field_images	id, field_id, image_url, is_primary	Foto lapangan
field_schedules	id, field_id, day_of_week, open_time, close_time, is_open	Jadwal operasional
field_slots	id, field_id, date, start_time, end_time, status, booking_id	Slot waktu per tanggal

3.2.3 Booking Service (booking_db)

Tabel	Kolom Utama	Keterangan
bookings	id, user_id, field_id, slot_id, date, start_time, end_time, total_price, status	Data booking
payments	id, booking_id, type, amount, status, payment_method, proof_url, paid_at	Riwayat pembayaran
booking_logs	id, booking_id, previous_status, new_status, note	Audit trail perubahan status

3.3 Paging dan Filtering

Endpoint	Parameter Paging	Parameter Filter
GET /service2/fields	page, per_page	category_id, status, location, min_price, max_price, search
GET /service2/fields/:id/slots	page, per_page	date, status, start_date, end_date
GET /service3/bookings	page, per_page	status, field_id, user_id (admin)

Contoh response dengan paging:

{ "success": true, "data": [...], "meta": { "total": 25, "per_page": 10, "current_page": 1, "last_page": 3 } }

BAB 4 - REST API PHP MVC (FIELD SERVICE)

4.1 Framework dan Struktur

Field Service dibangun menggunakan Laravel 11 dengan pola MVC (Model-View-Controller). Laravel dipilih karena kemampuannya dalam mengelola relasi database yang kompleks dan menyediakan fitur validasi input yang lengkap.

4.2 Pemisahan Layer MVC

Layer	File	Tanggung Jawab
Model	Field.php, FieldCategory.php, FieldSlot.php, FieldImage.php, FieldSchedule.php	Representasi data dan relasi antar tabel menggunakan Eloquent ORM
Controller	FieldController.php, FieldCategoryController.php, FieldSlotController.php	Logika bisnis, validasi input, dan pembentukan response JSON
Route	routes/api.php	Definisi endpoint dan mapping ke controller dengan middleware
Middleware	JwtVerify.php, GatewayOnly.php, RoleMiddleware.php	Autentikasi, otorisasi, dan pembatasan akses langsung

4.3 Komunikasi Inter-Service

Field Service dikonsumsi oleh Booking Service untuk operasi lock dan release slot. Ini merupakan implementasi komunikasi inter-service dalam arsitektur microservice. Booking Service mengirim request HTTP ke endpoint Field Service dengan menyertakan x-gateway-secret dan x-user-* headers.

Endpoint	Method	Dipanggil oleh	Fungsi
/api/slots/:id/lock	PATCH	Booking Service	Mengunci slot saat booking dibuat
/api/slots/:id/release	PATCH	Booking Service	Melepas slot saat booking dibatalkan

BAB 5 - AUTENTIKASI JWT

5.1 Implementasi JWT

Sistem mengimplementasikan JSON Web Token (JWT) untuk autentikasi stateless. JWT dipilih karena cocok dengan arsitektur microservice yang bersifat stateless.

Jenis Token	Masa Berlaku	Payload	Penyimpanan
Access Token	15 menit	id, email, role	Client (memory/header)
Refresh Token	7 hari	id	Database (tabel refresh_tokens)

5.2 Alur Autentikasi

1. User melakukan POST /auth/login dengan email dan password.
2. Server memverifikasi kredensial dan menghasilkan access token (15 menit) dan refresh token (7 hari).
3. Refresh token disimpan di database, access token dikirim ke client.
4. Client menyertakan access token di header Authorization: Bearer <token> untuk setiap request.
5. API Gateway memverifikasi signature JWT dan mengecek blacklist ke Auth Service.
6. Jika access token expired, client mengirim POST /auth/refresh dengan refresh token.
7. Saat logout, access token dimasukkan ke blacklist dan refresh token di-revoke.

5.3 Endpoint JWT

Endpoint	Method	Deskripsi
/service1/auth/login	POST	Login dan mendapatkan access token + refresh token
/service1/auth/refresh	POST	Mendapatkan access token baru menggunakan refresh token
/service1/auth/logout	POST	Invalidasi token (blacklist access token, revoke refresh token)
/service1/auth/validate-token	POST	Validasi token dan cek blacklist (digunakan API Gateway)

5.4 JWT Middleware

Setiap endpoint terproteksi diverifikasi melalui middleware, bukan manual per-endpoint. Terdapat dua pendekatan middleware:

Middleware	Digunakan di	Fungsi
jwtValidate.js	Auth Service (/validate-token)	Verifikasi signature JWT + cek blacklist database
jwtMiddleware.js (Auth & Booking)	Protected routes	Membaca x-user-* headers yang dikirim Gateway
JwtVerify.php (Field Service)	Protected routes Laravel	Membaca x-user-* headers yang dikirim Gateway
jwtVerify (Gateway)	Semua request ke Gateway	Verifikasi JWT + hit validate-token ke Auth Service

BAB 6 - OTORISASI OAUTH 2.0

6.1 Implementasi Google OAuth 2.0

Sistem mengimplementasikan OAuth 2.0 menggunakan Google sebagai provider sesuai paritas NIM genap. OAuth diimplementasikan menggunakan library passport-google-oauth20 dengan flow Authorization Code (bukan Implicit Flow yang sudah deprecated).

6.2 Alur OAuth 2.0 Authorization Code

8. User mengakses GET /service1/auth/oauth/google melalui browser.
9. Gateway meneruskan ke Auth Service, Passport.js meredirect ke halaman login Google.
10. User memilih akun Google dan menyetujui izin akses (email, profile).
11. Google meredirect ke callback URL dengan authorization code.
12. Auth Service menukar code dengan Google access token.
13. Auth Service mengambil data profil user dari Google API (nama, email, foto).
14. Jika user belum ada, dibuat otomatis dengan flag oauth_provider = 'google'.
15. Jika email sudah terdaftar manual, akun digabungkan dengan info OAuth.
16. Sistem menghasilkan JWT sendiri (access token + refresh token) dan mengembalikannya.

6.3 Data yang Disimpan dari Provider

Field	Sumber	Keterangan
name	profile.displayName	Nama lengkap dari Google
email	profile.emails[0].value	Email utama Google
profile_photo_url	profile.photos[0].value	URL foto profil Google
oauth_provider	Static 'google'	Nama provider OAuth
oauth_provider_id	profile.id	ID unik user di Google
is_oauth_user	Static TRUE	Flag bahwa user login via OAuth

6.4 Konfigurasi OAuth

Parameter	Nilai	Keterangan
Provider	Google OAuth 2.0	Sesuai paritas NIM genap
Endpoint	https://accounts.google.com/o/oauth2/v2/auth	Authorization endpoint Google
Scope	openid, email, profile	Izin data yang diminta
Flow	Authorization Code	Bukan Implicit Flow
Callback URL	http://localhost:3001/auth/oauth/google/callback	URL redirect setelah login

BAB 7 - API GATEWAY

7.1 Gambaran Umum

API Gateway berfungsi sebagai single entry point untuk seluruh service. Semua request dari client wajib melewati Gateway - akses langsung ke service dikembalikan dengan response 403 Forbidden menggunakan mekanisme x-gateway-secret.

7.2 Fitur API Gateway

Fitur	Implementasi	Keterangan
Routing	http-proxy-middleware	Meneruskan request ke service yang tepat berdasarkan path prefix
JWT Validation	jsonwebtoken + fetch ke validate-token	Verifikasi token sebelum diteruskan, termasuk cek blacklist
Rate Limiting	express-rate-limit	Maksimal 60 request per menit per IP
Gateway Secret	x-gateway-secret header	Memastikan service hanya dapat diakses melalui Gateway
User Context	x-user-* headers	Meneruskan data user terverifikasi ke service
Error Handling	Custom error handler	Menangani error koneksi ke service (ECONNREFUSED, timeout)

7.3 Peta Routing

Path Prefix	Service Tujuan	Port	Keterangan
/service1/*	Auth Service	3001	JWT + OAuth, strip /service1
/service2/*	Field Service	3002	Laravel, strip /service2, tambah /api
/service3/*	Booking Service	3003	Booking + payment, strip /service3

7.4 Public Endpoints (Tanpa JWT)

Method	Path	Keterangan
POST	/service1/auth/register	Registrasi user baru
POST	/service1/auth/login	Login dengan email/password
POST	/service1/auth/refresh	Refresh access token
GET	/service1/auth/oauth/google	Redirect ke Google OAuth
GET	/service1/auth/oauth/google/callback	Callback setelah login Google

Method	Path	Keterangan
POST	/service1/auth/validate-token	Validasi token (internal gateway)
GET	/service2/fields	Daftar lapangan (public)
GET	/service2/categories	Daftar kategori (public)

7.5 Mekanisme Rate Limiting

Gateway menerapkan rate limiting menggunakan library express-rate-limit dengan konfigurasi:

- Window: 60 detik (1 menit)
- Maximum request: 60 request per IP per window
- Response ketika limit tercapai: HTTP 429 Too Many Requests
- Header informasi limit disertakan dalam setiap response (RateLimit-Limit, RateLimit-Remaining)

7.6 Mekanisme Gateway Secret

Untuk memastikan service hanya dapat diakses melalui Gateway, setiap request yang diteruskan Gateway menyertakan header x-gateway-secret. Setiap service memvalidasi header ini sebelum memproses request:

- Request tanpa atau dengan x-gateway-secret yang salah dikembalikan dengan HTTP 403.
- Nilai GATEWAY_SECRET disimpan di file .env masing-masing komponen dan harus identik.
- Komunikasi inter-service (Booking ke Field) juga menyertakan header ini.

BAB 8 - STRUKTUR REPOSITORY DAN GITHUB

8.1 Struktur Repository

```
uts-pplos-b-2410511078/
├── README.md
├── gateway/
│   ├── middlewares/
│   │   └── authMiddleware.js
│   ├── node_modules/
│   ├── utils/
│   ├── .env
│   ├── app.js
│   ├── package-lock.json
│   └── package.json
├── services/
│   ├── auth-service/
│   ├── insurance-service/
│   └── document-ai-service/
├── docs/
│   ├── laporan-uts.pdf
│   └── arsitektur.png
└── postman/
    └── collection.json
```

8.2 Konvensi Commit

Repository mengikuti format Conventional Commits:

Prefix	Penggunaan	Contoh
feat:	Fitur baru	feat: add JWT login endpoint
fix:	Perbaikan bug	fix: token blacklist not working
docs:	Perubahan dokumentasi	docs: update README endpoint map
refactor:	Refactoring kode	refactor: extract catchAsync utility
test:	Penambahan test	test: add booking controller test
chore:	Konfigurasi/setup	chore: add dotenv to dependencies

BAB 9 - PENGUJIAN SISTEM

9.1 Metode Pengujian

Pengujian dilakukan menggunakan Postman dengan collection yang mencakup seluruh endpoint dari ketiga service. Setiap request memiliki test script yang memverifikasi status code dan struktur response.

9.2 Skenario Pengujian

9.2.1 Auth Service

Skenario	Method	Endpoint	Expected
Register user baru	POST	/service1/auth/register	201 Created
Register email duplikat	POST	/service1/auth/register	409 Conflict
Register field kosong	POST	/service1/auth/register	400 Bad Request
Login berhasil	POST	/service1/auth/login	200 OK + tokens
Login password salah	POST	/service1/auth/login	401 Unauthorized
Akses profil dengan token	GET	/service1/auth/me	200 OK + user data
Akses profil tanpa token	GET	/service1/auth/me	401 Unauthorized
Refresh token valid	POST	/service1/auth/refresh	200 OK + new tokens
Logout berhasil	POST	/service1/auth/logout	200 OK
Akses dengan blacklisted token	GET	/service1/auth/me	401 Unauthorized

9.2.2 Field Service

Skenario	Method	Endpoint	Expected
Get semua lapangan (public)	GET	/service2/fields	200 OK + pagination
Filter lapangan by status	GET	/service2/fields?status=active	200 OK + filtered
Buat lapangan (admin)	POST	/service2/fields	201 Created
Buat lapangan (non-admin)	POST	/service2/fields	403 Forbidden
Buat lapangan tanpa token	POST	/service2/fields	401 Unauthorized
Buat lapangan validasi error	POST	/service2/fields	422 Unprocessable
Buat slot waktu	POST	/service2/fields/:id/slots	201 Created

Skenario	Method	Endpoint	Expected
Buat slot konflik waktu	POST	/service2/fields/:id/slots	409 Conflict
Get lapangan tidak ada	GET	/service2/fields/invalid-id	404 Not Found

9.2.3 Booking Service

Skenario	Method	Endpoint	Expected
Buat booking baru	POST	/service3/bookings	201 Created
Buat booking slot sudah dipakai	POST	/service3/bookings	409 Conflict
Get booking milik sendiri	GET	/service3/bookings/:id	200 OK
Get booking milik user lain	GET	/service3/bookings/:id	403 Forbidden
Bayar DP	POST	/service3/bookings/:id/payments	201 Created
Bayar DP sudah dibayar	POST	/service3/bookings/:id/payments	409 Conflict
Bayar lunas	POST	/service3/bookings/:id/payments	201 Created
Batalkan booking	POST	/service3/bookings/:id/cancel	200 OK
Dashboard admin	GET	/service3/bookings/dashboard	200 OK + stats
Dashboard non-admin	GET	/service3/bookings/dashboard	403 Forbidden

9.2.4 Gateway - Akses Langsung ke Service

Skenario	Endpoint Langsung	Expected
Akses auth service langsung	http://localhost:3001/auth/login	403 Forbidden
Akses field service langsung	http://localhost:3002/api/fields	403 Forbidden
Akses booking service langsung	http://localhost:3003/bookings	403 Forbidden

BAB 10 - KESIMPULAN

10.1 Kesimpulan

Persyaratan	Status	Implementasi
Minimal 3 microservice + API Gateway	Selesai	Auth (3001), Field (3002), Booking (3003), Gateway (3000)
Database terpisah per service	Selesai	auth_db, field_db, booking_db
HTTP method dan response code semantik	Selesai	GET/POST/PUT/PATCH/DELETE, 200-500
Database relasional minimal 4 tabel	Selesai	Field Service: 5 tabel ter-relasi
Validasi input	Selesai	Laravel validator + custom AppError
Paging dan filtering	Selesai	Diterapkan di /fields dan /bookings
PHP MVC dengan framework	Selesai	Laravel 11 untuk Field Service
Komunikasi inter-service	Selesai	Booking Service ke Field Service (lock/release)
JWT access + refresh token	Selesai	15 menit access, 7 hari refresh
Endpoint refresh dan logout/blacklist	Selesai	POST /auth/refresh dan /auth/logout
JWT middleware	Selesai	Middleware global, bukan manual per-endpoint
OAuth 2.0 Google (NIM genap)	Selesai	passport-google-oauth20, Authorization Code flow
Pemetaan user OAuth ke lokal	Selesai	Auto-create dengan flag oauth_provider
Data profil dari provider	Selesai	Nama, email, foto profil disimpan
API Gateway single entry point	Selesai	http-proxy-middleware
JWT validation di gateway	Selesai	Verifikasi + cek blacklist ke auth-service
Rate limiting	Selesai	60 req/menit per IP dengan express-rate-limit
Dokumentasi peta routing	Selesai	Terdokumentasi di laporan dan README